

CSA0208-C Programming

Automobile Service Centre Management System

SUBMITTED BY:

HARISHA SK(192312311)

USHA RANI C(192312304)

THILAGAVATHI K(192312301)

1. Problem Statement

In many automobile service centres, vehicle service details are maintained manually using paper registers. This traditional method creates several difficulties such as loss of records, difficulty in searching vehicle history, calculation mistakes in bills, and time consumption during customer service. To overcome these limitations, an **Automobile Service Centre Management System** is developed using the C programming language. The application provides a digital platform for maintaining vehicle service records, calculating service charges, updating customer information, and generating service summaries. The system stores complete details about each serviced vehicle, including vehicle registration number, customer details, vehicle model, type of service performed, service date, spare parts cost, labour charges, and final billing amount. The application follows a menu-driven approach where service centre employees can perform different operations easily. The system also uses file handling techniques to save records permanently so that previous service information can be accessed whenever required. This project demonstrates the practical application of C programming concepts such as structures, pointers, arrays, functions, searching algorithms, sorting algorithms, and file management.

2. Objective

The main objective of developing the **Automobile Service Centre Management System** is to design and implement a computerized application that simplifies and automates the process of managing vehicle service records. The system aims to replace the traditional manual register method with a digital solution that provides accurate, efficient, and organized storage of customer and vehicle information. The application is designed to maintain complete service details such as vehicle number, customer name, vehicle model, service type, service date, spare parts cost, labour cost, and total service charges. It helps service centre employees to quickly register new vehicles, search existing records, update service information, calculate bills automatically, and generate service summaries. Another important objective is to apply C programming concepts such as structures, pointers, functions, arrays, sorting algorithms, and file handling to solve a real-world management problem. The use of pointers allows efficient updating of records, while file handling ensures that service data is stored permanently and can be retrieved whenever required. The system also focuses on improving data accuracy by implementing validation techniques to handle duplicate vehicle numbers, invalid costs, incorrect dates, and storage limitations. Additionally, the project aims to analyze algorithm performance and understand the trade-off between simplicity and efficiency while selecting suitable methods for searching and sorting service records. Overall, the objective of this project is to create a reliable, user-friendly, and efficient automobile service management system that

reduces manual effort, minimizes errors, and improves the overall service centre management process.

3. Requirements and Environment Used

Hardware Requirements

The hardware requirements for executing this project are:

Component	Specification
Processor	Intel i3/i5 or equivalent
RAM	Minimum 4 GB
Storage	100 MB available space
Input Device	Keyboard and Mouse
Output Device	Monitor

Software Requirements

Software	Details
Operating System	Windows/Linux
Programming Language	C
Compiler	GCC Compiler
IDE	CodeBlocks / Visual Studio Code
File Format	Binary File (.dat)

4. Design / Proposed Solution

The proposed **Automobile Service Centre Management System** follows a modular design approach. The complete application is divided into separate functional modules to improve program organization, reduce complexity, and make the system easier to develop, test, and maintain. Each module performs a specific task related to vehicle service management.

Module 1: Vehicle Registration Module

The Vehicle Registration Module is used to add new vehicle service records into the system. It collects important details such as vehicle registration number, customer name, vehicle model, service type, service date, parts cost, and labour cost. Before storing the record, the system verifies whether the vehicle number already exists to prevent duplicate entries and maintain accurate service information.

Module 2: Vehicle Search and Update Module

The Search and Update Module helps users retrieve and modify existing vehicle service records. The vehicle number is used as the search key to quickly locate customer and service

details. This module reduces manual searching time and provides previous service history. It also allows users to update information such as service type, customer details, parts cost, and labour charges. Pointers to structures are used for efficient modification of records.

Module 3: Billing and Cost Calculation Module

The Billing Module automatically calculates the total service charge for each vehicle. The total cost is calculated by adding the spare parts cost and labour cost using the formula **Total Service Cost = Parts Cost + Labour Cost**. This module helps to generate accurate bills, reduces calculation errors, and improves the efficiency of the billing process.

Module 4: Sorting and Report Generation Module

The Sorting and Report Generation Module organizes service records based on service cost. It helps the service centre identify expensive repairs, analyze service revenue, and manage records effectively. This module also generates daily service summaries, including the total number of vehicles serviced, total income generated, and high-cost service details.

Module 5: File Management and Error Handling Module

The File Management Module is responsible for storing and retrieving service records permanently using file handling techniques. It saves data into a file and loads previous records when the program starts, preventing data loss and maintaining service history. This module also handles errors such as duplicate vehicle numbers, invalid costs, incorrect dates, file opening failures, and maximum record capacity limitations to ensure reliable system operation.

5. Algorithm / Pseudocode

Algorithm for Registration:

START

Accept vehicle details from user

Check whether vehicle number already exists

IF vehicle exists

Display duplicate record message

ELSE

Store new vehicle details

Calculate service cost

Save record

END IF

STOP

Algorithm for Searching:

START

Enter vehicle number

Compare entered number with stored records

IF match found

Display vehicle details

ELSE

Display record not found

STOP

Algorithm for Billing:

START

Read parts cost

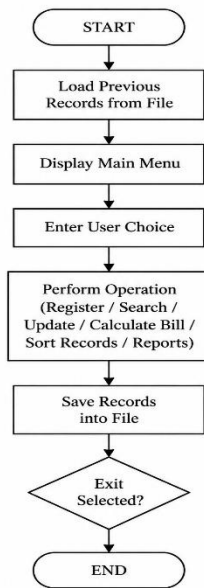
Read labour cost

Calculate:

Total = Parts Cost + Labour Cost
Display total amount

STOP

FLOW CHART



6. Implementation / Source Code Explanation

The program implementation uses a structure named ServiceRecord.

```
struct ServiceRecord
```

```
{
```

```
char vehicleNo[20];
```

```
char customerName[50];
```

```
char model[30];
```

```
char serviceType[30];
```

```
char date[15];
```

```
float partsCost;
```

```
float labourCost;
```

```
float totalCost;
```

```
};
```

The structure stores complete information about a vehicle service.

The program contains separate functions for:

- Adding records
- Searching records
- Updating records
- Calculating bills
- Sorting records
- Saving files
- Loading files

This modular approach makes the program easier to debug and maintain.

7. Sorting Algorithm Analysis

The program uses Bubble Sort to arrange service records according to cost.

Working Principle

Bubble sort repeatedly compares adjacent records and exchanges them if they are in the wrong order.

Example:

Before sorting: 5000 2000 8000

After sorting: 2000 5000 8000

Performance Comparison

Algorithm	Time Complexity
Bubble Sort	$O(n^2)$
Merge Sort	$O(n \log n)$
Quick Sort	$O(n \log n)$

For a small automobile service centre, bubble sort is acceptable. For large companies with thousands of records, faster algorithms should be implemented.

8. Error Handling Mechanism

A reliable application must be capable of handling unexpected situations and preventing incorrect data entry or system failures. In the Automobile Service Centre Management System, various error-handling techniques are implemented to ensure the accuracy and stability of the application. The system handles duplicate vehicle numbers by searching existing records before registering a new vehicle. If the entered vehicle number is already available in the records, the system displays a message indicating that the vehicle already exists and cancels the registration.

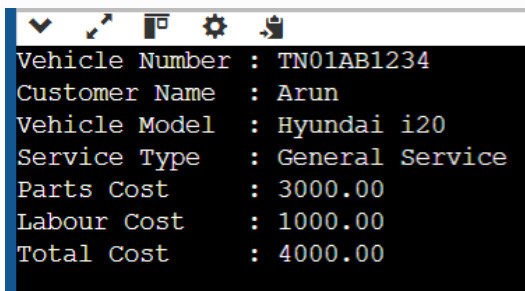
process. This prevents duplicate entries and maintains unique vehicle identification. The program also performs cost validation to ensure that the parts cost and labour cost entered by the user are valid. The system checks that all cost values are greater than or equal to zero. If the user enters a negative value, the program rejects the input and displays an error message, preventing incorrect billing calculations.

Date validation is another important error-handling feature of the system. The application checks whether the entered service date follows the correct format by validating the day range between 1 and 31, the month range between 1 and 12, and the proper year format. Invalid dates are rejected to maintain accurate service records. The system also handles maximum record capacity limitations to prevent array overflow. Before inserting a new service record, the program checks whether the current record count has reached the maximum allowed limit (MAX_RECORDS). If the storage capacity is full, the system displays a message indicating that no additional records can be added. These error-handling mechanisms improve the reliability, accuracy, and safety of the application.

9. Test Cases and Result

Test Case 1: Normal Service

OUTPUT:



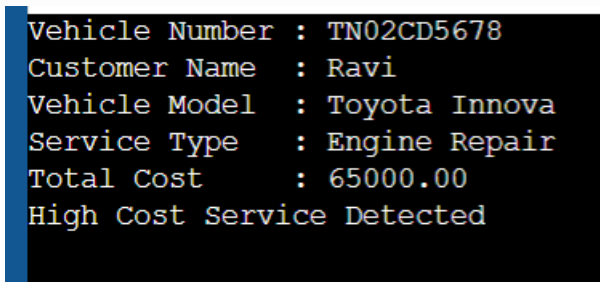
```
Vehicle Number : TN01AB1234
Customer Name  : Arun
Vehicle Model  : Hyundai i20
Service Type   : General Service
Parts Cost     : 3000.00
Labour Cost    : 1000.00
Total Cost     : 4000.00
```

Test Case 2: High-Cost Service

Purpose:

To test whether the system identifies expensive service records.

OUTPUT:



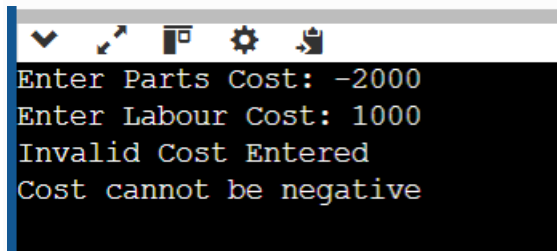
```
Vehicle Number : TN02CD5678
Customer Name   : Ravi
Vehicle Model   : Toyota Innova
Service Type    : Engine Repair
Total Cost      : 65000.00
High Cost Service Detected
```

Test Case 3: Invalid Input Condition

Purpose:

To test error handling for negative service costs.

OUTPUT:



```
Enter Parts Cost: -2000
Enter Labour Cost: 1000
Invalid Cost Entered
Cost cannot be negative
```

Final code:

```
#include<stdio.h>

#include<string.h>

#define MAX 100

struct ServiceRecord
{
    char vehicleNo[20];
    char customerName[30];
    char model[30];
    char serviceType[30];
    float partsCost;
    float labourCost;
    float totalCost;
};

void calculateCost(struct ServiceRecord *s)
{
```



```

        s->totalCost = s->partsCost + s->labourCost;
    }

int searchVehicle(char vehicleNo[], struct ServiceRecord s[], int count)
{
    for(int i=0;i<count;i++)
    {
        if(strcmp(s[i].vehicleNo,vehicleNo)==0)
            return i;
    }
    return -1;
}

int main()
{
    struct ServiceRecord s;

    printf("Enter Vehicle Number: ");
    scanf("%s",s.vehicleNo);

    printf("Enter Parts Cost: ");
    scanf("%f",&s.partsCost);

    printf("Enter Labour Cost: ");
    scanf("%f",&s.labourCost);

    calculateCost(&s);

    printf("Total Service Cost = %.2f",s.totalCost);

    return 0;
}

```

10. Analysis and Discussion

The developed system successfully automates automobile service centre operations. The use of C programming concepts improves efficiency and provides a strong foundation for real-

world applications. Structures improve data organization, pointers improve modification efficiency, and file handling ensures permanent storage. The system reduces paperwork, improves accuracy, and saves time compared to manual methods. The implementation of searching and sorting techniques helps in managing service records effectively and provides quick access to required information. The validation mechanisms ensure that incorrect data such as duplicate vehicle numbers, negative costs, and invalid dates are prevented from being stored. The modular design of the program improves readability, debugging, and future maintenance of the application. The system can be further enhanced by integrating a database, user authentication, and a graphical interface to support large-scale automobile service centres. Overall, the project demonstrates how programming concepts can be effectively applied to develop a practical and reliable management system.

11. Conclusion

The **Automobile Service Centre Management System** developed using C programming successfully provides an efficient solution for managing vehicle service records digitally. The system replaces the traditional manual register method by automating important operations such as vehicle registration, searching, updating service details, calculating bills, sorting records, generating summaries, and storing data permanently using file handling. The use of structures helps in organizing vehicle and customer information effectively, while pointers allow efficient updating and manipulation of records. Error-handling techniques ensure that invalid inputs, duplicate vehicle numbers, incorrect costs, file access problems, and record limitations are managed properly. The project demonstrates the practical implementation of important C programming concepts such as functions, structures, pointers, arrays, algorithms, and file operations in a real-world application. The selected sorting approach provides a simple and understandable solution for arranging service records based on cost, while also allowing analysis of efficiency for larger datasets. Overall, the system reduces manual effort, improves accuracy, saves time, and provides a reliable platform for automobile service centre management. Future enhancements can include database integration, user authentication, graphical user interface, and online service booking features to make the system more advanced and suitable for large-scale service centres.

12. References

1. E. Balagurusamy, *Programming in ANSI C*, 8th Edition, McGraw Hill Education, 2019.
2. Yashavant Kanetkar, *Let Us C*, 19th Edition, BPB Publications, 2023.
3. Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*, 2nd Edition, Prentice Hall, 1988.
4. Herbert Schildt, *C: The Complete Reference*, 4th Edition, McGraw Hill Education, 2000.
5. Reema Thareja, *Programming in C*, 2nd Edition, Oxford University Press, 2016.
6. GCC (GNU Compiler Collection) Documentation, GNU Project. Available at: <https://gcc.gnu.org/documentation/>

7. C Programming Language Documentation, TutorialsPoint.
Available at: <https://www.tutorialspoint.com/cprogramming/>
8. GeeksforGeeks – C Programming Concepts and Data Structures.
Available at: <https://www.geeksforgeeks.org/c-programming-language/>
9. Microsoft Visual Studio Code Documentation (for C programming environment setup).
Available at: <https://code.visualstudio.com/docs>
10. ISO/IEC 9899:2018 – Programming Languages – C Standard, International Organization for Standardization (ISO).